PYTHON CORE • MODULE 5

Exception Handling (try / except)

Comprehensive Practice Tasks • 25 Hands-on Lab Exercises

TOTAL EXERCISES	CORE CONCEPTS	REAL-WORLD LABS
25 Tasks Across 8 Sections	**try, except, else, finally, raise**	**ATM, Quiz, Cart, Marksheet**

DIFFICULTY LEVELS: ☆ Easy — Fundamentals · ☆☆ Medium — Applied Logic · ☆☆☆ Hard — Complex & Real World

TOPICS COVERED IN THIS PRACTICE SET

1. Basic try / except syntax
2. Catching specific exceptions (ValueError, ZeroDivisionError, etc.)
3. Inspecting errors with `as e`
4. Multi-except hierarchies & general fallback
5. The `else` clause for clean success execution
6. The `finally` block for guaranteed cleanup
7. Complete 4-part architecture (try / except / else / finally)
8. Nested exception handling & loop validation workflows

SECTION 1 BASIC TRY / EXCEPT

Tasks 1 – 3

Task 01 Basic Integer Input Conversion

☆ Easy

Ask the user to enter a number and convert it to an integer using `int()`.
If the user inputs something non-numeric (e.g. `"hello"`), catch the error and print:
`"Invalid input! Please enter a number."`

🧪 **Test with:** `"hello"`, `"abc"`, `"10"`

Task 02 ZeroDivisionError Handling

☆ Easy

Ask the user for two numbers. Divide the first by the second.
If the second number is `0`, catch the `ZeroDivisionError` and print:
`"Cannot divide by zero!"`

🧪 **Test with:** `10, 0` and `10, 2`

Task 03 Safe List Index Access (IndexError)

☆ Easy

Create a list: `fruits = ["apple", "banana", "cherry"]`.
Ask the user to enter an index number and try to print `fruits[index]`.
If the index is out of range, catch `IndexError` and print:
`"That index doesn't exist!"`

🧪 **Test with:** `0`, `5`, `-10`

SECTION 2 SPECIFIC EXCEPTIONS & "AS E"

Tasks 4 – 7

Task 04 Dictionary Key Lookup (KeyError)

☆ Easy

Create a dictionary: `student = {"name": "Ravi", "age": 20}`.
Ask the user to enter a key name (like `"name"`, `"age"`, `"marks"`) and try to print the value.
If the key doesn't exist, catch `KeyError` and print:
`"Key not found!"`

🧪 **Test with:** `"name"`, `"marks"`

Task 05 Incompatible Types (TypeError)

☆ Easy

Ask the user for their name.
Try to do: `result = name + 10`
Catch the `TypeError` and print:
`"Cannot add text and number together!"`

Task 06 Capturing Error Message Details with "as e"

☆☆ Medium

Ask the user to enter a number and try to convert it to `int`.
If it fails, catch `ValueError as e` and print:
`print(f"Error details: {e}")`

🧪 **Test with:** `"abc"` → should show the actual Python error message

Task 07 "as e" with IndexError

☆☆ Medium

Create a list of 3 city names. Ask user for an index.
If `IndexError` occurs, print the actual error message using `as e`.
Format: `"Oops! {e}"`

SECTION 3 MULTIPLE EXCEPT BLOCKS

Tasks 8 – 10

Task 08 Two except Blocks

☆☆ Medium

Ask the user for a number, convert to int. Then divide 100 by that number.
Handle BOTH:

- `ValueError` (if they type text)
- `ZeroDivisionError` (if they type 0)

Print a different message for each.

🧪 **Test with:** `"abc"`, `0`, `5`

Task 09 Three except Blocks with General Fallback

☆☆ Medium

Create a list: `prices = ["100", "abc", "200"]`. Ask user for an index.
Try to:

1. Access `prices[index]`
2. Convert it to `int`
3. Print the value

Handle: `IndexError`, `ValueError`, and a general `Exception`.

🧪 **Test with:** `0`, `1`, `5`

Task 10 Multiple Exceptions in a Calculator

☆☆ Medium

Ask user for: `number1`, `operator` (+, -, *, /), `number2`.
Perform the calculation.
Handle:

- `ValueError` (non-numeric input)
- `ZeroDivisionError` (dividing by 0)
- `Exception` (anything else, like invalid operator)

🧪 **Test with:** `10 / 0`, `"abc" + 5`, `10 / 2`

SECTION 4 THE ELSE BLOCK

Tasks 11 – 13

Task 11 try / except / else Flow

☆☆ Medium

Ask the user for their age (convert to int).

- If conversion succeeds (no error): in the `else` block, print `"You are {age} years old!"`
- If conversion fails: print `"That's not a valid age!"`

🧪 **Test with:** `25`, `"twenty"`

Task 12 else with Division

☆☆ Medium

Ask for two numbers.

- In `try`: convert both to int and divide.
- In `except`: handle `ValueError` and `ZeroDivisionError`.
- In `else`: print `"Division successful! Result = {result}"`

(The else block should ONLY run when there is NO error.)

Task 13 else with List Access

☆☆ Medium

Create a list: `colors = ["red", "green", "blue"]`. Ask user for an index.

- In `try`: access `colors[index]`.
- In `except`: handle `IndexError`.
- In `else`: print `"You picked: {color}"` and also print `"This color has {len} letters"`.

💡 **Best Practice:** Extra processing in `else` keeps your `try` block compact and focused.

SECTION 5 THE FINALLY BLOCK

Tasks 14 – 16

Task 14 try / except / finally Execution

☆☆ Medium

Ask the user for a number. Try to divide 100 by that number.
In `except` : handle `ZeroDivisionError` and `ValueError` .
In `finally` : print `"Calculation attempt finished."`
The finally message should appear whether there was an error or NOT.

🧪 **Test with:** `0` , `5` , `"abc"`

Task 15 finally as Cleanup Guarantee

☆☆ Medium

Simulate opening a connection: `print("Opening database connection...")` .
In `try` : ask user for a number and divide 50 by it.
In `except` : handle errors.
In `finally` : print `"Closing database connection..."` .
The connection must ALWAYS close — even if there's an error!

Task 16 Complete Pipeline: try / except / else / finally

☆☆☆ Hard

Ask the user for a password:

- In `try` : check if `len(password) >= 8` — if not, raise:
  `raise ValueError("Password must be at least 8 characters")`
- In `except` : catch `ValueError as e` , print the error.
- In `else` : print `"Password accepted!"`
- In `finally` : print `"Password check complete."`

🧪 **Test with:** `"abc"` , `"mypassword123"`

SECTION 6 NESTED TRY / EXCEPT

Tasks 17 – 18

Task 17 Nested: Processing a List of Heterogeneous Items

☆☆☆ Hard

Given this data:

```
numbers = ["10", "abc", "20", "0", "30"]
```

Loop through each item:

- **Inner try:** convert to int, then divide 100 by it.
- **Inner except:** handle `ValueError` and `ZeroDivisionError` separately.
- **Outer try:** wraps the whole loop as a safety net.
- Print ✅ for success and ❌ for each error.

📋 **Expected Output:**
✅ 100/10 = 10.0
❌ 'abc' is not a valid number
✅ 100/20 = 5.0
❌ Cannot divide by zero
✅ 100/30 = 3.33

Task 18 Nested: Student Grades Calculation ☆☆☆ Hard

Given:

```
students = [
    {"name": "Aman", "score": "85", "total": "100"},
    {"name": "Neha", "score": "xyz", "total": "100"},
    {"name": "Kiran", "score": "70", "total": "0"},
]
```

- **Outer try:** process all students.
- **Inner try:** convert `score` and `total` to int, calculate percentage.
- **Inner except:** handle `ValueError` (bad score) and `ZeroDivisionError` (total=0).
- Print each student's result or error message clearly.

SECTION 7 LOOPS + TRY/EXCEPT (INPUT VALIDATION) Tasks 19 – 21

Task 19 Keep Asking Until Valid ☆☆ Medium

Use a `while` loop to keep asking the user for a number.
If they enter something invalid, print an error and ask again.
Only break out of the loop when they enter a valid integer.
After the loop, print: `"You entered: {number}"`

Task 20 Validate Age with Range Check ☆☆ Medium

Keep asking for age until the user enters a valid integer that is ALSO between `1` and `120`.
Handle `ValueError` for non-numeric input.
Use `else` block to check the range — if out of range, continue the loop.
If valid, break and print: `"Your age: {age}"`

Task 21 Collect 5 Valid Numbers ☆☆☆ Hard

Ask the user to enter 5 numbers one by one.
If they enter something invalid, show an error and DON'T count it.
Keep asking until you have exactly 5 valid numbers.
At the end, print all 5 numbers and their sum.

> 💡 **Hint:** Use a list to store valid numbers, loop while `len(list) < 5`.

SECTION 8 REAL-WORLD SCENARIOS Tasks 22 – 25

Task 22 ATM Withdrawal Simulator ☆☆☆ Hard

Starting balance = `10000`.
In a loop:

- Ask user how much to withdraw (0 to exit).
- Handle `ValueError` if they type text.
- If `amount > balance`: print `"Insufficient funds!"`
- If `amount <= 0` (but not exit): print `"Enter a positive amount!"`
- If valid: subtract from balance, print remaining balance.

Use `finally` inside the loop to always print `"--- Transaction complete ---"`.

Task 23 Shopping Cart Checkout System

⭐⭐⭐ Hard

Given a product catalog:

```
catalog = {"pen": 10, "notebook": 50, "bag": 500, "bottle": 150}
```

In a loop:

- Ask the user for a product name and quantity.
- Handle `KeyError` if the product doesn't exist.
- Handle `ValueError` if quantity is not a number.
- If valid, add to a cart list and running total.
- Type `"done"` to finish.

At the end, print the receipt (all items + total).
Use `else` block to add item only when no error occurred.

Task 24 Mark Sheet Processor with Custom Range Validation

⭐⭐⭐ Hard

Given:

```
raw_marks = [
    ("Ravi", "85"), ("Priya", "abc"), ("Amit", "92"),
    ("Sneha", "-10"), ("Kiran", "78"), ("Neha", "110"),
]
```

Process each student:

- Convert marks to int (handle `ValueError`).
- Check if marks are between 0 and 100 — if not, raise:
  `raise ValueError("Marks out of range")`
- Determine grade: `>=90 A`, `>=75 B`, `>=60 C`, else `D`

Use nested `try/except`. Print ✅ for valid entries, ❌ for errors.
At the end, print how many passed and how many had errors.

Task 25 Mini Quiz Game with Exception Handling

⭐⭐⭐ Hard

Create a quiz with 5 questions (stored as a list of tuples):

```
questions = [
    ("What is 5 + 3?", 8),
    ("What is 10 / 2?", 5),
    ("What is 7 * 6?", 42),
    ("What is 100 - 37?", 63),
    ("What is 2 ** 5?", 32),
]
```

For each question:

- Ask the user for their answer.
- In `try`: convert answer to int.
- In `except`: print `"Invalid answer! Skipping..."` and move to next.
- In `else`: check if answer matches → update score.
- In `finally`: print `"Question {n} done."`

At the end, print `"You scored {score}/5"`.
Wrap everything in an outer `try/except` for safety.